

Bulao.

From zero to submitted in five days.

"Bolo, aur kaam ho jaye."

Version 2.0 of the technical build guide for Bulao — the voice-first, Urdu-native AI service orchestrator — updated for the full official Challenge 2 specification. Now covers six agents, dynamic pricing, six-factor matching, the service-quality loop, and the dispute & escalation workflow.

Architecture, prerequisites, day-by-day procedure, every Antigravity prompt to paste, full system prompts for all six agents, mock data schemas, demo script, and the pre-submission checklist.

CONTENTS

01	System overview & stack	07	Day 4 · Service quality + dispute
02	Day-zero prerequisites	08	Day 5 · Demo, docs, submit
03	The Antigravity workflow	A	The six agent system prompts
04	Day 1 · Foundation	B	Mock data schemas
05	Day 2 · Intent & Discovery	C	Antigravity prompt library
06	Day 3 · Ranking, Booking, UI	D	Failure modes & checklist

01 · SYSTEM OVERVIEW

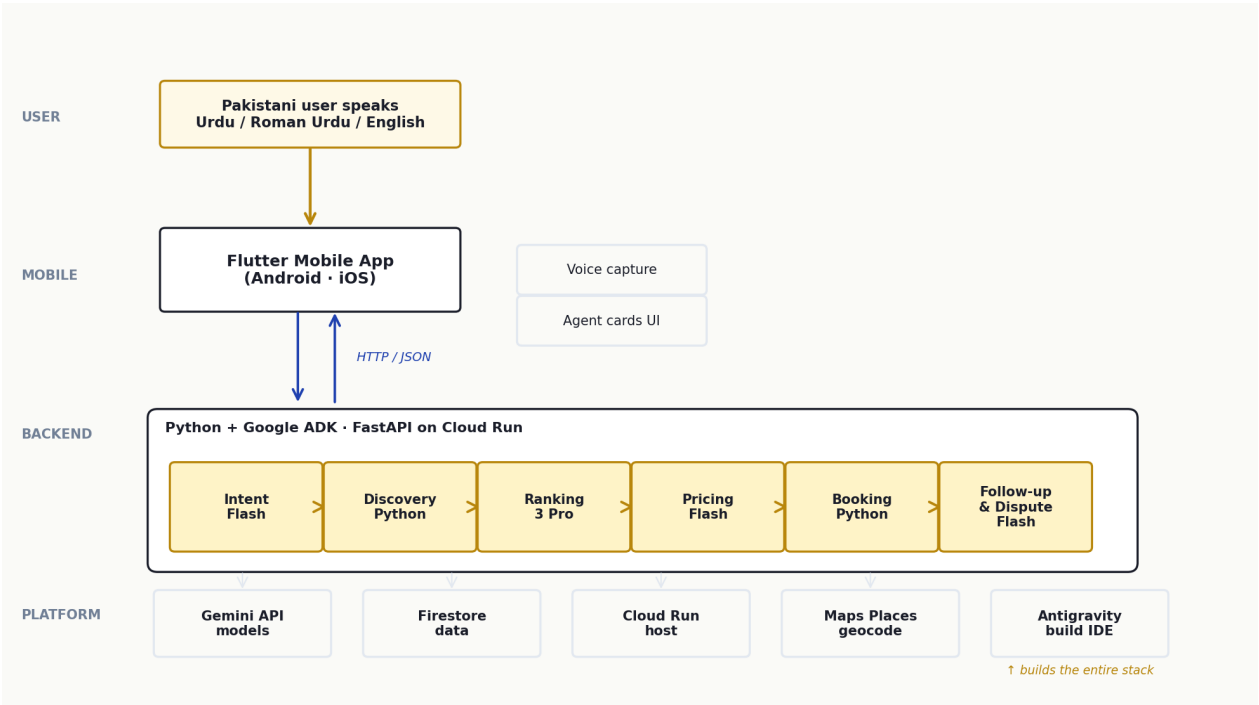
What you're actually building.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy. A user opens the app, presses a big button, speaks in Urdu, Roman Urdu, or English, and within ninety seconds has watched **six AI agents** extract their intent (with a confidence score and complexity tag), find providers nearby, rank them across six factors with visible reasoning, produce a transparent price breakdown, book a slot, track service through to completion, and — if anything goes wrong — resolve the dispute fairly. The magic is that the agents are *visible* on screen as they work. That visibility is the agentic moneyshot the rubric rewards.

WHAT'S NEW IN v2.0

This version of the playbook is rebuilt against the full official Challenge 2 specification (released after v1 was drafted). The architecture grows from five agents to six with the addition of a dedicated **Pricing Agent**. Significant new scope: **(a)** confidence scoring and clarification questions on the Intent Agent; **(b)** job-complexity classification (basic / intermediate / complex); **(c)** a six-factor matching algorithm on the Ranking Agent; **(d)** dynamic pricing with a transparent line-item breakdown; **(e)** a service-quality loop with en-route tracking and completion checklist; **(f)** a dispute & escalation workflow with classified resolutions; **(g)** scheduling intelligence (conflict prevention, alternate slots).

The system at a glance



Three layers: the Flutter mobile app captures voice and renders the agent-card UI; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the entire stack — not part of the runtime, but the source of your 25%-weighted Agent Trace deliverable.

The six agents



Tech stack at a glance

Layer	Choice	Why
Mobile	Flutter 3.x (Dart)	One codebase ships Android + iOS; rendering is good enough for animated agent cards.
Backend language	Python 3.11	ADK is most mature in Python; team can hire help fast if stuck.
Agent framework	Google ADK	First-class Gemini integration; SequentialAgent and tool calling out of the box.
Server framework	FastAPI + uvicorn	Async, easy to deploy, plays well with ADK.
Models	Gemini Flash + Gemini 3 Pro	Flash for cheap/fast steps (intent, ranking heuristics); Pro for complex reasoning.
Voice	Gemini 3 Pro audio input	Direct audio→text in one model call; no separate STT service needed.
Data store	Firestore (native mode)	Free tier covers a hackathon; latency in asia-south1 is fine.
Hosting	Cloud Run (asia-south1)	Auto-scales to zero, free credits cover the demo, deploys in 90 seconds.
Maps	Google Maps Places API	Real provider lookup if there's time; mock JSON fallback otherwise.
IDE / build agent	Google Antigravity	Mandatory by the hackathon rules; produces the Agent Trace deliverable.

Team roles



Four roles, one creative lead and three developers. If you only have three people, merge Mobile and Infra/Trace; the creative lead's responsibilities are non-delegable.

02 · DAY-ZERO PREREQUISITES

Install this before the build starts.

Every teammate completes this checklist before May 15. Doing this on Day 1 wastes four hours of build time. The accounts and credits can take up to a day to provision, so don't leave them for the morning of.

On every laptop

Google Antigravity

Download from [antigravity.google/download](https://antigravity.google.com/download) and run the installer for your OS. Launch it, sign in with a personal Gmail (work accounts may have restrictions), and complete the first-run wizard. Choose **Agent-assisted development** mode — not Agent-driven (too autonomous for safety) and not Review-driven (too slow for hackathon pace). Verify Gemini 3 Pro is available as your primary model.

Flutter SDK (3.x)

```
# macOS / Linux
git clone https://github.com/flutter/flutter.git -b stable ~/flutter
export PATH="$PATH:$HOME/flutter/bin"
flutter doctor
flutter doctor --android-licenses # accept all

# Windows
# Download .zip from flutter.dev, extract to C:\src\flutter, add to PATH
```

Python 3.11+ with pip

```
python3 --version # must show 3.11 or higher
python3 -m pip --version # confirm pip is installed

# If you need to install Python:
# macOS: brew install python@3.11
# Ubuntu: sudo apt install python3.11 python3.11-venv python3-pip
# Windows: download from python.org, check 'Add to PATH'
```

Google Cloud CLI (gcloud)

```
# Install: https://cloud.google.com/sdk/install
gcloud --version
gcloud init # sign in, pick project
gcloud auth login
gcloud auth application-default login # for SDK auth
```

Supporting tools

```
node --version    # 20.x or higher (some Flutter tooling needs it)
git --version     # any recent version
docker --version  # for testing the backend container locally

# Android Studio for: Android emulator, APK signing for the final build
# VS Code as a backup IDE if Antigravity acts up
```

Accounts and API access

Google AI Studio API key (for Gemini)

Go to aistudio.google.com, sign in, click **Get API key**, create a new key in a fresh project named **bulao-hackathon**. Copy it now — you'll paste it into the backend's .env file on Day 1. The free tier limits are generous enough for the entire build week.

Google Cloud project (for Cloud Run, Firestore, Maps)

In Cloud Console, create a project called **bulao-hackathon**. Enable these APIs: Cloud Run, Firestore, Maps Places API, Cloud Build, Artifact Registry, Cloud Scheduler (for the Follow-up Agent), and Secret Manager. Set the billing account to the hackathon credits (redeem the link from the InnoVista email). Create a Firestore database in **native mode**, region **asia-south1** (Mumbai — closest to Pakistan).

Redeem the hackathon credits

The email from InnoVista contained a link — redeem it within Cloud Console → Billing. The credits are time-limited; do this before May 15. The team's free Antigravity Gemini quota is separate and doesn't need redeeming.

Shared resources

- **GitHub repo:** bulao-hackathon (private). Invite all teammates. Set up branch protection on main, require PR reviews from at least one teammate.
- **Shared Google Drive folder** for Antigravity screenshots, demo footage, design assets. Sub-folders for each build day.
- **WhatsApp group** for fast coordination — faster than Slack for a 5-day sprint.
- **Notion or shared doc** for the running 50-example Urdu utterance dataset and the demo storyboard.
- **One designated demo phone** — a real Android device (not a simulator) that will be used for the demo video and the regional finals presentation. Charge it.

If you finish the prerequisites checklist by Day -1 (May 14), you start Day 1 with two extra hours. That's two extra hours of polish on the demo — which is exactly where points are won.

03 · THE ANTIGRAVITY WORKFLOW

How to use the IDE so you actually score the 25%.

Antigravity is not just a place to write code. It's an agent-orchestration platform where multiple AI agents work in parallel on different parts of your codebase, producing verifiable Artifacts — plans, diffs, screenshots, browser recordings. Most teams will use it like a fancy autocomplete and lose two-thirds of the 25%. Use it the way it was designed and you bank those points.

Two views, two modes of work

Editor View

A VS Code-style IDE with an agent sidebar. Use this when you're hands-on with a single file — tab completions, inline commands, refactors. This is where you go when you need to read the code, not when you need to build a feature.

Manager View

The agent-first surface. You describe what you want at a high level, Antigravity spawns an agent in its own workspace, the agent produces a **Plan Artifact**, you approve or refine it, the agent executes — writing code, running terminal commands, testing in a browser. You can spawn **multiple agents in parallel**, each on a different workspace. This is where the 25%-weighted screenshots come from. Live in Manager View this week.

Day-1 settings to configure

Open Antigravity Settings — Agent panel. Set each of the following deliberately:

Mode	Agent-assisted development	You stay in control; AI helps with safe automations.
Artifact Review Policy	Asks for Review	Forces a Plan Artifact before code is written — this IS the screenshot you submit.
Terminal Command Auto Execution	Request Review	Don't let agents run arbitrary shell commands without you seeing them first.
Enable Terminal Sandbox	ON	Limits the blast radius of terminal commands.
Default model	Gemini 3 Pro	Use Pro for hard tasks; switch to Flash for quick edits.

Workflow per task

Every meaningful piece of work follows the same rhythm. Internalise this on Day 1 and every teammate runs the same playbook for five days:

-
- | | | |
|-----------|-----------------------|---|
| 01 | Frame the task | Write the request in natural language with a clear deliverable. "Build the Intent Agent system prompt and integrate it into the FastAPI orchestrator" beats "Make the intent thing work." |
|-----------|-----------------------|---|
-
- | | | |
|-----------|-------------------------------|--|
| 02 | Read the Plan Artifact | Antigravity produces a Plan with steps, files it'll touch, commands it'll run. Screenshot this. If the plan looks wrong, ask the agent to revise before it starts coding. |
|-----------|-------------------------------|--|
-
- | | | |
|-----------|----------------------------|---|
| 03 | Approve and execute | Watch the agent work. It writes code, runs the terminal, launches the browser. Don't interrupt unless something is going seriously wrong. |
|-----------|----------------------------|---|
-
- | | | |
|-----------|-------------------------|--|
| 04 | Review Artifacts | The agent produces diffs, screenshots, test logs. Save the meaningful ones to your Antigravity Trace folder. A browser recording of the agent testing your app is gold. |
|-----------|-------------------------|--|
-
- | | | |
|-----------|-------------------------|---|
| 05 | Test it yourself | Run the result on your phone or in your terminal. Verify it actually works — agents can be confidently wrong. |
|-----------|-------------------------|---|
-
- | | | |
|-----------|-------------------------------|---|
| 06 | Save to Knowledge Base | Antigravity has a Knowledge Base where you can save context, decisions, snippets the agent should remember. Save your Intent Agent prompt here, the API endpoint contract, the design tokens. Future agents inherit this context. |
|-----------|-------------------------------|---|
-

Daily screenshot checklist

Take all of these every single build day. Save them to docs/antigravity-trace/day{N}/. By Day 5 you'll have a stockpile to edit down into a highlight reel for the submission.

- **One Manager View screenshot** showing at least three agents working in parallel on different parts of the project. This is the marquee shot for the rubric.
- **One Plan Artifact** from a non-trivial feature — ideally something where the agent broke down a vague request into a clear step plan.
- **One browser recording** of an Antigravity agent testing the running app. The agent literally drives the browser; that's the headline capability of the platform.
- **One diff or code review** screenshot showing an Artifact the agent produced and a teammate's comment guiding a revision.
- **One Knowledge Base entry** showing useful context being saved for future agents — a system prompt, a design token list, an API contract.

Three prompting patterns that work

Pattern A: the parallel kickoff

On Day 1, open three Manager View workspaces and paste a different scaffolding prompt in each. All three agents run simultaneously. By the time you've finished your coffee, you have a working Flutter app, a running FastAPI backend, and a deployed Cloud Run service.

Pattern B: the iterative refinement

Don't try to one-shot complex features. Start with: "Create a stub Intent Agent that always returns the same JSON." Verify it runs. Then: "Now make it actually parse the user input using Gemini Flash with this system prompt: ..." Verify again. Then: "Add Urdu/Roman Urdu test cases to tests/intent_agent_test.py and make sure they pass." Three small wins beat one big stuck.

Pattern C: the browser-verified handoff

Ask the agent to *verify* the work in the browser. "After implementing the AgentCard widget, run the Flutter app, navigate to the home screen, trigger a sample intent, and take screenshots showing the card animating through pending → active → done states." The browser recording of this is one of your best Artifacts.

Think of Antigravity as four junior engineers working for you in parallel. Your job is to give clear briefs, review Plan Artifacts before they code, and document the good work for the submission package. The 25% rubric weight is yours to lose.

04 · DAY 1 (MAY 15)

Foundation: scaffolding, deployment, hello-world.

By the end of today, three things have to be true: the Flutter app runs on a real phone showing the home screen, the FastAPI backend is deployed to Cloud Run and responds to /orchestrate with stub data, and the repo has the trace folder structure and screenshot conventions locked in. If you don't hit these by 8pm, you're working until midnight — and waking up tired on Day 2.

Morning · admin (60–90 min)

- **Submit Challenge 2** on the selection form. This is final — cannot be changed later.
- Every teammate confirms their prerequisites checklist is done (Section 02).
- Create the GitHub repo bulao-hackathon, clone to every laptop.
- Open Antigravity, configure the Day-1 settings from Section 03.
- Confirm roles: Product/Urdu, Mobile, Agents, Infra/Trace.
- Open Antigravity Manager View on every developer's laptop — you're about to spawn three parallel agents.

Afternoon · parallel scaffolding

In Antigravity Manager View, the three developers each open a new workspace and paste the relevant prompt below. The agents work simultaneously. Total wall time for this section: about two hours, most of it watching agents work.

PROMPT A · MOBILE TRACK · PASTE IN ANTIGRAVITY MANAGER VIEW

Create a new Flutter project called bulao_app inside the mobile/ subdirectory of this repo. Use Flutter stable 3.x targeting Android 8+ and iOS 13+.

Configure these things:

1. Material 3 theme. Primary color #1a1d29 (deep ink), accent #b8860b (gold), background #fafaf7 (warm white). Use the default Roboto font for now; we will add Urdu fonts on Day 3.
2. Add these dependencies to pubspec.yaml: go_router (^14.x), http (^1.x), record (^5.x) for voice capture, audioplayers (^6.x), google_maps_flutter (^2.x), flutter_animate (^4.x), lottie (^3.x), intl (^0.19.x), provider (^6.x) for simple state management.
3. Create the screen structure with go_router:
 - HomeScreen at "/" — centered press-and-hold microphone button (using a CircleAvatar with a microphone icon), an Urdu greeting "Aapko kis cheez ki zaroorat hai?", and a small "Tap and hold to speak" hint below.
 - ProcessingScreen at "/process" — shows a vertical stack of 5 AgentCard widgets that activate one by one.
 - ConfirmationScreen at "/confirm" — shows the booking receipt and a WhatsApp-style confirmation chat bubble.

4. Create a reusable widget `AgentCard` in `lib/widgets/agent_card.dart` that takes:

- String `agentName` (e.g. "Intent Agent")
- `AgentStatus` `status` (enum: pending, active, done)
- String `title` (the agent's output headline)
- Widget `child` (the agent's output body)
- Color `accentColor` (defaults to gold #b8860b)

Visual behavior: when pending, the card is faded with no border. When active, it animates a subtle pulse and shows the accent color as a left rule. When done, a checkmark fades in on the right and the card settles into a static rest state.

5. Create the API service at `lib/services/api.dart` with two methods:

- `Future> orchestrate(String text, {String? userLocation})`
- `Future> confirmBooking(String bookingId)`

For now, both methods should return realistic mock JSON after a 500ms simulated delay. We will wire to the real backend on Day 2.

6. Set up simple state management using a `ChangeNotifier` called `OrchestrationState` that holds the current agent statuses and outputs. Provide it via the provider package.

7. Add a basic `theme.dart` with `TextStyles` for headline, body, caption.

Once everything is in place, run the app on the iOS simulator and the Android emulator. Take screenshots of the HomeScreen on both. Save those screenshots in `docs/antigravity-trace/day1/mobile/`.

PROMPT B · BACKEND TRACK · PASTE IN ANTIGRAVITY MANAGER VIEW

Create a Python backend at backend/ that hosts the 5-agent system for Bulao. Use Python 3.11.

1. Create a virtual environment at backend/.venv. Add a requirements.txt with: google-adk, fastapi, uvicorn[standard], google-genai, pydantic, python-multipart, firebase-admin, pillow, reportlab, python-dotenv.

2. Create backend/main.py — a FastAPI app exposing:

- POST /orchestrate — body {"text": str, "user_location": Optional[Dict[str, float]]}, returns the full pipeline result with each agent's output.

- POST /book — body {"booking_request": {...}}, returns {"booking_id": str, "receipt_url": str, "confirmation_message": str}.

- GET /status/{booking_id} — returns the booking record.

- GET /health — returns {"status": "ok", "version": "0.1.0"} for Cloud Run health checks.

3. Set up the ADK agent skeleton at backend/agents/. Create six files, each with a placeholder that returns hardcoded JSON. We will replace placeholders one agent per day.

- intent_agent.py — defines an ADK LlmAgent named "intent_agent" using model "gemini-flash-latest".

System prompt is just "Return the user input as JSON with a 'service_type' field for now."

- discovery_agent.py — a plain Python function discover(intent: dict) -> List[Dict] that loads backend/data/providers.json and returns the first 5 entries for now.

- ranking_agent.py — an ADK LlmAgent named "ranking_agent" using "gemini-3-pro-latest". Placeholder prompt: "Pick the first provider and explain why in one sentence."

- pricing_agent.py — an ADK LlmAgent named "pricing_agent" using "gemini-flash-latest". Placeholder: returns a stub price breakdown with base_visit_fee, distance_fee, and total.

- booking_agent.py — a plain Python function book(provider_id: str, intent: dict, price: dict) -> Dict that returns a stub booking record including service-quality fields (status: "confirmed", en_route_at: null, in_progress_at: null, completed_at: null).

- followup_agent.py — a plain Python function with two entry points: schedule_followup(booking_id) for reminders/check-ins, and handle_dispute(booking_id, dispute_type, user_message) for the dispute path.

4. Create backend/agents/orchestrator.py that sequences the six agents. For Day 1, use plain Python function calls in sequence. Day 2 we'll convert to ADK's SequentialAgent.

5. Create backend/data/providers.json with 100 fake Pakistani service providers. Each provider object has: id (string, "prov_001" to "prov_100"), name (Pakistani Urdu/English names), service_categories (array of strings from this list: "plumber", "electrician", "ac_technician", "geyser_technician", "carpenter", "painter", "beautician", "tutor", "appliance_repair", "gas_leak_specialist"), specializations (array of sub-skills, e.g. ["inverter_ac", "split_ac"] for AC techs, ["bridal", "party"] for beauticians), lat (decimal between 33.4 and 33.7), lng (decimal between 72.9 and 73.2), neighborhood (F-6, F-7, F-8, F-10, F-11, G-7, G-9, G-10, G-11, G-13, Bahria Town, DHA, Saddar, Pindi Cantt), rating (decimal 3.5–5.0), completed_jobs (integer 5–200), completed_jobs_by_neighborhood (object mapping neighborhood name to count), on_time_score (decimal 0.5–1.0; reliability metric), cancellation_rate (decimal 0.0–0.3), review_recency_days (integer 1–180 since most recent review), risk_score (decimal 0.0–1.0; computed from cancellations + complaints; lower is better), current_workload (decimal 0.0–1.0; how booked-up they are right now), available_slots (array of ISO datetimes for the next 7 days), base_visit_fee_pkr (integer 300–800), rate_per_hour_pkr (integer 500–2000), phone ("++923XX-XXXXXXX" format), gender ("male" or "female"), years_experience (integer 1–25), languages (subset of ["urdu", "english", "punjabi"]), verified (boolean), strikes (integer 0–3; dispute history). The richer schema supports six-factor matching, dynamic pricing, and dispute handling — do not skip any fields.

6. Create a Dockerfile that uses python:3.11-slim, copies requirements, installs them, copies the app, and runs uvicorn main:app --host 0.0.0.0 --port 8080. Cloud Run sets the PORT env variable; respect it.

7. Add `backend/.env.example` listing required environment variables: `GOOGLE_API_KEY`, `GOOGLE_CLOUD_PROJECT`, `FIRESTORE_DATABASE` (defaults to `"(default)"`), `GEMINI_MODEL_FLASH=gemini-flash-latest`, `GEMINI_MODEL_PRO=gemini-3-pro-latest`.

8. Write a basic test at `backend/tests/test_orchestrator.py` that calls the orchestrator with a hardcoded input and asserts all six agent outputs are present in the response.

Verify by running: `cd backend && pip install -r requirements.txt && uvicorn main:app --reload`. Hit `/health` with `curl`. Hit `/orchestrate` with `curl -X POST -d '{"text": "I need a plumber"}'`. Save the terminal output as a screenshot to `docs/antigravity-trace/day1/backend/`.

PROMPT C · INFRA & TRACE TRACK · PASTE IN ANTIGRAVITY MANAGER VIEW

Set up the deployment pipeline, demo infrastructure, and Antigravity trace folder for Bulao.

1. Confirm the Google Cloud project bulao-hackathon exists with billing linked to hackathon credits. Enable these APIs via gcloud: `run.googleapis.com`, `firestore.googleapis.com`, `cloudbuild.googleapis.com`, `artifactregistry.googleapis.com`, `places.googleapis.com`, `secretmanager.googleapis.com`, `cloudscheduler.googleapis.com`.
 2. Create an Artifact Registry repo named "bulao" in region asia-south1.
 3. Create a Secret Manager secret named "gemini-api-key" containing the team's Gemini API key from AI Studio. Grant the default Cloud Run service account access to read it.
 4. Create `backend/cloudbuild.yaml` that:
 - Builds the Docker image from `backend/Dockerfile` tagged with `$SHORT_SHA`
 - Pushes it to `asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:$SHORT_SHA`
 - Deploys to Cloud Run in asia-south1, service name "bulao-backend", with environment variable `GOOGLE_API_KEY` sourced from the secret, `GOOGLE_CLOUD_PROJECT` set to bulao-hackathon, min instances 0, max instances 5, memory 1Gi.
 5. Create `scripts/deploy.sh` that runs: `gcloud builds submit --config backend/cloudbuild.yaml backend/`
 6. Run `scripts/deploy.sh` once now to deploy the stub backend. Verify the Cloud Run service URL responds to `/health`. Note the URL — paste it into a file at `backend/.cloud_run_url` for the mobile team to use.
 7. Create the Antigravity Trace folder structure at `docs/antigravity-trace/` with subfolders:
 - `day1/`, `day2/`, `day3/`, `day4/`, `day5/`
 - `plan-artifacts/`
 - `manager-view/`
 - `browser-recordings/`
 - `knowledge-base/`
 - `final-highlights/`
 Each subfolder should have a placeholder `README.md` explaining what kind of screenshot goes there.
 8. Create `README.md` at the repo root with these sections (placeholders for now): Project Overview, The 5-Agent Architecture, How Antigravity Was Used, Tools & APIs, Setup & Running Locally, Deployment, Demo Video Link, Team. We will fill these on Day 5.
 9. Create `docs/demo-storyboard.md` with timestamped shot descriptions at 0:00, 0:15, 0:30, 0:45, 1:00, 1:30, 2:00, 2:30, 3:00, 3:30. Each entry has columns: timestamp, shot description, what's on screen, voiceover line (Urdu and English).
 10. Set up GitHub Actions: on push to main, run flake8 on `backend/`, dart analyze on `mobile/`, and a build check (no actual deploy from Actions — deploys go through gcloud for visibility).
- Verify: cloud run URL works, all folders exist, README is in place with structure. Take a screenshot of the Cloud Run console showing the deployed service.

End-of-day verification

Before anyone goes home, run these tests together. If any fail, fix them tonight.

SANITY CHECK — TERMINAL COMMANDS

```
# Backend works locally
curl http://localhost:8080/health
# → {"status": "ok", "version": "0.1.0"}

# Backend works on Cloud Run
curl https://bulao-backend-<hash>-as.a.run.app/health
# → {"status": "ok", "version": "0.1.0"}

# Orchestrate returns all 5 agent outputs (stub)
curl -X POST https://bulao-backend-<hash>-as.a.run.app/orchestrate \
  -H 'Content-Type: application/json' \
  -d '{"text": "plumber chahiye"}'
# → {"intent": {...}, "discovery": [...], "ranking": {...},
#    "booking_ready": {...}, "followup_planned": {...}}
```

Flutter app: press-and-hold the mic button. The app navigates to /process, calls the (mock) API, and shows 5 agent cards stepping through pending → active → done in sequence. The output is hardcoded but the animation flows.

Antigravity trace assets to capture today

- Manager View screenshot showing the three parallel agents (Mobile, Backend, Infra) all running simultaneously.
- Plan Artifact screenshot from at least one of the three scaffolding tasks — the step plan the agent produced before writing code.
- Browser recording (or screen capture) of an Antigravity agent running the Flutter app in a simulator and verifying the home screen renders.
- Terminal screenshot showing the Cloud Run deploy completing and the URL being printed.

If Day 1 ends with hello-world working end-to-end — phone → Cloud Run → mock response → animated cards — you're ahead of 80% of teams. Don't try to build the Intent Agent tonight; sleep wins.

05 · DAY 2

Intent and Discovery agents go live.

Today the system gets its first real intelligence. The Intent Agent learns to understand Urdu and Roman Urdu; the Discovery Agent learns to find providers. By 8pm, typing a Pakistani-flavored sentence into the Flutter app produces a real list of relevant providers on screen.

Morning · the 50-example utterance dataset

Before touching code, the Product/Urdu lead spends ninety minutes drafting fifty real Pakistani utterances. This dataset is the source of truth for prompt tuning, the regression suite for the Intent Agent, and the basis for the demo voiceover. Without it, Urdu support is a marketing claim — with it, it's a moat.

How to write the dataset

Save it as `backend/data/intent_examples.jsonl` — one JSON object per line. Each entry has the raw user text and the expected structured output. Mix freely across all the language patterns Pakistanis actually use:

FORMAT · INTENT_EXAMPLES.JSONL (FIRST 5 OF 50)

```
{"input": "Mujhe G-13 mein kal subah AC theek karwana hai", "expected": {"service_type": "ac_technician", "location": "G-13", "time_window": "tomorrow_morning", "urgency": "normal", "job_complexity": "intermediate", "gender_preference": "any", "confidence_min": 0.85}}
{"input": "Abhi jaldi se plumber bhejo, geyser leak ho raha hai", "expected": {"service_type": "plumber", "location": null, "time_window": "now", "urgency": "emergency", "job_complexity": "basic", "raw_notes": "geyser leak", "confidence_min": 0.7, "needs_clarification": "location"}}
{"input": "Inverter AC ka PCB jal gaya, koi specialist chahiye DHA mein", "expected": {"service_type": "ac_technician", "location": "DHA", "time_window": "flexible", "urgency": "high", "job_complexity": "complex", "specialization_hint": "inverter_ac", "raw_notes": "PCB burnt", "confidence_min": 0.9}}
{"input": "Need a female beautician for bridal makeup this Friday in DHA Karachi", "expected": {"service_type": "beautician", "location": "DHA Karachi", "time_window": "this_friday", "urgency": "normal", "job_complexity": "complex", "specialization_hint": "bridal", "gender_preference": "female", "raw_notes": "bridal makeup", "confidence_min": 0.9}}
{"input": "F-10 mein electrician chahiye, AC ki wiring kharab hai", "expected": {"service_type": "electrician", "location": "F-10", "time_window": "flexible", "urgency": "normal", "job_complexity": "intermediate", "raw_notes": "AC wiring", "confidence_min": 0.85}}
```

Coverage requirements for your 50 examples

- At least 20 in Urdu/Roman Urdu, at least 10 in English, the rest code-switched.
- Cover all 10 service categories from `providers.json`.
- **Complexity coverage:** 15 basic, 25 intermediate, 10 complex examples. Complex examples should mention specializations (inverter AC, PCB repair, bridal, etc.).
- Cover at least 8 different Islamabad/Rawalpindi/Karachi neighborhoods.
- Mix of urgency levels: 8 emergencies, 25 normal, 17 flexible.

- Include 8 ambiguous examples (vague location, vague service, mixed signals) where the model should return confidence < 0.7 with a clarification_question populated.
- Include 5 edge cases: typos, mid-sentence corrections, double-service requests.

Afternoon · build the Intent Agent

The Agents teammate replaces the stub `intent_agent.py` with the real implementation. Full system prompt is in Appendix A; here's the Antigravity prompt that builds the agent around it:

PROMPT · BUILD THE REAL INTENT AGENT

Replace the stub `intent_agent.py` at `backend/agents/intent_agent.py` with a real implementation that uses Gemini Flash to extract structured intent from user text.

1. Use google-adk's `LlmAgent` class with model "gemini-flash-latest".
2. The system prompt is in `backend/agents/prompts/intent_agent_system.md` — load it from disk at module import time. (Create that file too; I will paste the full prompt into it after you generate the structure.)
3. The agent must return valid JSON conforming to this Pydantic schema:

```
class Intent(BaseModel):
    service_type: Literal["plumber", "electrician", "ac_technician", "geyser_technician", "carpenter", "painter",
        "beautician", "tutor", "appliance_repair", "gas_leak_specialist"]
    location: Optional[str]
    city: str = "Islamabad"
    time_window: str
    urgency: Literal["emergency", "high", "normal", "flexible"]
    gender_preference: Literal["male", "female", "any"] = "any"
    job_complexity: Literal["basic", "intermediate", "complex"]
    specialization_hint: Optional[str] = None # e.g. "inverter_ac" if mentioned
    confidence: float # 0.0 to 1.0; the model's confidence in this extraction
    clarification_question: Optional[str] = None # Urdu question if confidence < 0.7
    budget_range: Optional[Tuple[int, int]] = None
    raw_notes: Optional[str] = None
```

4. After Gemini returns text, parse with `json.loads`, then validate with `Intent.model_validate`. On parse failure, retry once with a temperature of 0 and a "Return ONLY valid JSON" reminder. On second failure, return a fallback intent with `service_type` inferred from keyword matching against the user text and `confidence=0.3` + `clarification_question="Aap ne kya kaam karwana hai?"`.

5. Wire the Intent Agent into the orchestrator at `backend/agents/orchestrator.py`: it runs first, its output becomes the input to the Discovery Agent. The `/orchestrate` endpoint returns the validated Intent in its response under "intent".

6. If `confidence < 0.7`, the orchestrator should NOT proceed to Discovery automatically. Instead it returns the intent + `clarification_question` to the mobile app, which renders the question as a tappable Urdu chip below the agent card. The user's response then re-runs the Intent Agent with the augmented context.

7. Write tests at `backend/tests/test_intent_agent.py` that load `intent_examples.jsonl` and assert: a) `service_type` accuracy $\geq 90\%$, b) `job_complexity` accuracy $\geq 85\%$, c) `confidence > 0.7` for unambiguous examples, d) `clarification_question` is populated when `confidence < 0.7`.

8. Add a CLI helper at `backend/scripts/eval_intent.py` that runs the agent over the dataset and prints accuracy + a confusion matrix on `service_type` AND on `job_complexity`. Useful for iterating on the prompt.

Once running, eval the agent and report the accuracy numbers. If `service_type` accuracy is below 90% or complexity below 85%, surface the failing examples so we can refine the system prompt.

After Antigravity scaffolds this, the Product/Urdu lead pastes the actual system prompt (Appendix A) into `backend/agents/prompts/intent_agent_system.md`, runs `python backend/scripts/eval_intent.py`, and

iterates on the prompt until accuracy crosses 92%. Expect 2–4 hours of prompt iteration. Save each version of the prompt with a git commit so you can roll back.

Late afternoon · the Discovery Agent

Discovery is the simplest of the six agents because it doesn't use an LLM. It takes structured intent and returns matching providers from the database. The Antigravity prompt:

PROMPT · BUILD THE DISCOVERY AGENT

Implement the Discovery Agent at backend/agents/discovery_agent.py. This is plain Python — no LLM. It does filtering, scheduling-conflict detection, and alternate-slot suggestions.

1. Define a function `discover(intent: Intent, user_location: Optional[LatLng] = None) -> DiscoveryResult`.
2. Load `providers.json` once at module import (cache it). For each request:
 - a) Filter to providers where `intent.service_type` is in their `service_categories`.
 - b) If `intent.specialization_hint` is set (e.g. "inverter_ac"), prefer (don't exclude) providers whose specializations list contains it.
 - c) If `intent.location` is set, filter to providers in that neighborhood OR within 5km of the geocoded neighborhood center.
 - d) If `intent.gender_preference` is "female" or "male" (not "any"), filter to matching providers.
 - e) Complexity filter: if `intent.job_complexity` is "complex", require `provider.years_experience >= 5` AND a relevant specialization. For "intermediate", require `years_experience >= 2`. "Basic" has no filter.
 - f) Compute `distance_km` for each remaining provider using haversine.
 - g) Compute `availability_status` by checking `provider.available_slots` against `intent.time_window` AND applying a 30-minute travel buffer (provider needs to be free 30 min before AND after the slot).
3. Scheduling intelligence — if fewer than 3 candidates remain after filters, run the alternate-slot path:
 - a) Take the top 5 providers (ignoring availability) and find their next available slot.
 - b) Return these as "alternates" with a note explaining why they weren't first-choice (e.g. "Earliest available is 11:30am instead of 10am").
 - c) The mobile UI shows these as a separate "Available later" section.
4. Return `DiscoveryResult`:


```
class DiscoveryResult(BaseModel):
    candidates: List[ProviderCandidate] # up to 8, ordered by Discovery's rough score
    alternates: List[ProviderCandidate] # 0-3, with reschedule reasons
    no_match_reason: Optional[str] # set if zero results, e.g. "no female AC tech available within 5km at this time"
```
5. `ProviderCandidate` fields (rich, for downstream agents): `id`, `name`, `service_categories`, `specializations`, `distance_km`, `neighborhood`, `rating`, `completed_jobs_in_area`, `on_time_score`, `cancellation_rate`, `review_recency_days`, `risk_score`, `current_workload`, `availability_status`, `next_slot` (ISO), `base_visit_fee_pkr`, `rate_per_hour_pkr`, `gender`, `years_experience`, `languages`, `verified`, `phone_masked`, `alternate_slot_reason` (only on alternates).
6. Sort candidates by a rough pre-rank score (this is just for the top-8 cutoff; the real ranking happens next): `availability_status weight + rating + (1 - risk_score) + (1 - current_workload)`. Top 8 only.
7. Define a small neighborhoods table mapping each neighborhood name to a (lat, lng) center, used when `user_location` is None. Cover all 14 neighborhoods in `providers.json`.
8. Wire into the orchestrator: Discovery Agent runs after Intent Agent. Its output is included in /orchestrate response under "discovery".

9. Add tests at backend/tests/test_discovery_agent.py. Cover: a) exact neighborhood match, b) emergency requests prioritize availability, c) gender filter works, d) complexity filter excludes junior providers for complex jobs, e) alternate-slot path triggers when fewer than 3 candidates after time-window filter, f) empty result returns no_match_reason explaining why.

10. Add an endpoint GET /providers/preview that returns a sample provider list for the mobile team to mock against.

Wire the mobile app to the real backend

By 6pm, the Mobile teammate replaces the stub api.dart with real HTTP calls to the Cloud Run backend. The flow on the device: user types or speaks → /orchestrate POST → backend returns intent + discovery → mobile renders the first two agent cards with real data. Cards 3-5 still show stub data; we replace them tomorrow.

End-of-day verification

- Intent Agent eval shows $\geq 92\%$ accuracy on service_type and $\geq 85\%$ on job_complexity over the 50-example dataset.
- confidence is > 0.7 for unambiguous examples and < 0.7 for the 8 ambiguous ones, with a sensible Urdu clarification_question.
- Discovery Agent returns sensible provider lists for at least 10 sample intents and triggers the alternate-slot path when fewer than 3 candidates are available.
- Complexity filter excludes junior providers for complex jobs.
- Flutter app calls real /orchestrate, renders Intent + Discovery cards with real data, falls back to stubs for cards 3–6.
- Antigravity trace folder has Day 2 screenshots: prompt iteration history, eval output, browser recording of the agent testing the Discovery endpoint.

If your Intent Agent fails on a real Urdu sentence at the regional finals, you lose. Spend the time on the dataset. The Urdu eval set is the most important file in the repo this week.

06 · DAY 3

Ranking, Pricing, Booking, and the UI that sells it.

Day 3 is when the product starts to feel real. **Three** LLM-powered agents come online today: Ranking reasons across six factors about which provider to pick, the new Pricing Agent produces a transparent line-item quote, and Booking generates a beautiful receipt the user keeps. By 9pm tonight, a typed Urdu sentence produces intent → discovery → ranking → price → booking end-to-end — the demo's spine.

Morning · the Ranking Agent (six-factor)

Ranking is the agent whose output is most visible to the user, so its quality matters disproportionately. It must score candidates across the six factors mandated by the rubric: distance/travel time, availability, rating + review recency, reliability/on-time score, skill specialization match, and risk score. Its system prompt (Appendix A) tells it to score, pick a top recommendation, and explain in natural Urdu and English why. The Antigravity build prompt:

PROMPT · BUILD THE RANKING AGENT

Implement the Ranking Agent at backend/agents/ranking_agent.py using Gemini 3 Pro.

1. Use google-adk's LlmAgent with model "gemini-3-pro-latest". Temperature 0.4 for natural reasoning variation.
2. The system prompt is at backend/agents/prompts/ranking_agent_system.md. The agent receives the validated Intent and the DiscoveryResult (candidates + alternates). It must return JSON conforming to:

```
class FactorScore(BaseModel):
    factor: Literal["distance", "availability", "rating_recency", "on_time_reliability", "specialization_match", "risk"]
    score: float # 0.0 to 1.0
    note: str # one-line explanation

class CandidateScore(BaseModel):
    candidate_id: str
    factor_scores: List[FactorScore] # exactly 6 entries
    weighted_total: float
    weight_profile_used: str # "emergency" | "high" | "normal" | "flexible"

class Ranking(BaseModel):
    recommended_id: str
    top_three_ids: List[str]
    all_scores: List[CandidateScore]
    reasoning_english: str # 1-2 sentences, MUST reference specific numbers
    reasoning_urdu: str # natural Urdu, same content
    tradeoffs: Optional[str] # honest comparison with #2 and #3
    confidence: Literal["high", "medium", "low"]
```

3. Use these weight profiles based on intent.urgency:
 - emergency: distance 0.30, availability 0.30, rating_recency 0.10, on_time 0.20, specialization 0.05, risk 0.05
 - high: distance 0.20, availability 0.25, rating_recency 0.15, on_time 0.20, specialization 0.10, risk 0.10
 - normal: distance 0.15, availability 0.10, rating_recency 0.25, on_time 0.20, specialization 0.20, risk 0.10
 - flexible: distance 0.10, availability 0.05, rating_recency 0.30, on_time 0.15, specialization 0.30, risk 0.10
4. Validate recommended_id is one of the candidates. Each reasoning string MUST contain at least one digit. Each candidate must have exactly 6 factor_scores. Retry once on validation failure.
5. The reasoning_english and reasoning_urdu MUST reference at least 3 of the 6 factors with specific numbers (e.g. "1.4km away, 4.8 rating from 47 jobs in your area, 98% on-time"). Generic reasoning like "best option for you" is unacceptable.
6. Wire into the orchestrator after Discovery. Output appears in /orchestrate response under "ranking".
7. Add tests at backend/tests/test_ranking_agent.py covering: a) emergency intent uses emergency weight profile, b) reasoning contains 3+ specific numbers, c) top_three_ids has exactly 3 entries when candidates >= 3, d) Urdu reasoning is detectably Urdu, e) factor_scores list has exactly 6 entries per candidate.
8. Update the mobile API service to render the Ranking Card with: the top pick highlighted, a small "Why?" button that expands to show the six-factor breakdown as a horizontal bar chart, reasoning in Urdu by default with a "translate" button.

Late morning · the Pricing Agent (new in v2)

The Pricing Agent is new in v2 of the playbook, added to comply with the rubric's dynamic-pricing requirement. It sits between Ranking and Booking. Its output is a transparent line-item quote the user sees and approves *before* confirming the booking. This is the agent that turns the rubric's "price disagreement" failure mode into a non-issue — the user agreed to the math upfront.

PROMPT · BUILD THE PRICING AGENT

Implement the Pricing Agent at backend/agents/pricing_agent.py using Gemini Flash.

1. Use google-adk's LlmAgent with model "gemini-flash-latest", temperature 0.2. The pricing logic itself is mostly deterministic; the LLM is used for the natural-language explanation.
2. The system prompt is at backend/agents/prompts/pricing_agent_system.md (Appendix A.4). It receives the intent, the recommended ProviderCandidate, and a market_demand score (0-1; the team can fake this as 0.65 for the demo). It returns JSON conforming to:

```
class PriceLineItem(BaseModel):
    label_english: str
    label_urdu: str
    amount_pkr: int # negative for discounts
    kind: Literal["base", "addon", "multiplier", "discount", "surge"]
```

```

class PriceQuote(BaseModel):
    line_items: List[PriceLineItem]
    subtotal_pkr: int
    estimated_total_pkr: int
    estimated_range_pkr: Tuple[int, int] # range_low <= total <= range_high
    explanation_english: str
    explanation_urdu: str
    quote_id: str
    expires_at: str # ISO datetime, 15 minutes from now

```

3. Pricing logic (deterministic, computed in Python BEFORE asking the LLM to write the explanation):

- base_visit_fee = provider.base_visit_fee_pkr
- estimated_hours by complexity: basic 1.0, intermediate 1.5, complex 2.5
- estimated_service_fee = provider.rate_per_hour_pkr * estimated_hours
- distance_fee = max(0, distance_km - 3) * 50 PKR (3km free)
- complexity_multiplier: basic 1.0, intermediate 1.25, complex 1.6
- urgency_surcharge: emergency +500, high +200, normal 0, flexible -100
- surge_factor: market_demand <= 0.5 → 1.0; (0.5, 0.8] → 1.1; > 0.8 → 1.2
- loyalty_discount: first_booking 0, otherwise -50 to -200 based on tier
- subtotal = (base_visit_fee + distance_fee + estimated_service_fee) * complexity_multiplier
- with_urgency = subtotal + urgency_surcharge
- with_surge = with_urgency * surge_factor
- total = with_surge + loyalty_discount
- Round to nearest 10 PKR
- range = (total * 0.9, total * 1.15); ensure range is at least 250 PKR wide

4. The LLM's job: take the computed line_items and totals, and produce a natural Urdu + English explanation (1-2 sentences each). Must mention surge_factor explicitly if > 1.0 ("demand zyada hai isliye thora surge"). Must mention the welcome discount if loyalty_discount < 0.

5. quote_id format: "QT-{booking_id_prefix}-{6 random chars}". Quote expires 15 minutes from generation; if the user takes longer to confirm, the Pricing Agent re-runs.

6. Wire into the orchestrator after Ranking. Output appears in /orchestrate response under "pricing". The /orchestrate endpoint returns the full pipeline result; /book accepts the user's accepted quote_id and proceeds.

7. Mobile integration: between the Ranking Card and the Confirm button, show a Pricing Card that lists every line_item with its label in Urdu and the amount. The estimated_range is shown prominently ("Estimate PKR 2,260 – 2,890"). A small "Why this price?" link expands the explanation. The Confirm button is greyed out until the user has seen the price (small UX detail; counts for transparency).

8. Add tests at backend/tests/test_pricing_agent.py covering: a) complex jobs cost more than basic, b) emergency adds surcharge, c) surge_factor applied correctly, d) first booking has no loyalty discount, e) all line_items have both Urdu and English labels, f) range is always wider than 250 PKR.

Midday · the Booking Agent

Booking now takes the confirmed price quote as input. It writes the booking record, generates the receipt PDF (now showing the agreed line-item breakdown), drafts the WhatsApp-style confirmation, and initializes the service-quality lifecycle (status fields for en-route, in-progress, completed — the actual transitions happen on Day 4). The receipt is the demo's hero asset; take it seriously.

PROMPT · BUILD THE BOOKING AGENT + RECEIPT PDF

Implement the Booking Agent at backend/agents/booking_agent.py with three jobs.

1. Function book(provider_id: str, intent: Intent, accepted_quote_id: str, user_info: UserInfo) -> Booking that:
 - a) Validates the quote_id is current (not expired). If expired, return an error so the mobile app can re-run /orchestrate.
 - b) Generates a booking_id of format "BUL-{YYYYMMDD}-{6 random uppercase chars}".
 - c) Writes a record to Firestore at collection "bookings" with all relevant fields: booking_id, provider_id, user_id, service_type, location, scheduled_time (parsed from intent.time_window), urgency, job_complexity, status="confirmed", lifecycle (object with: confirmed_at, en_route_at: null, arrived_at: null, in_progress_at: null, completed_at: null, cancelled_at: null), accepted_quote (full PriceQuote snapshot for audit), expected_price_range, actual_price_pkr: null (filled after service), contact_phone (full, no longer masked).
 - d) Generates a receipt PDF using reportlab. Visually beautiful, A6 phone-sized. Includes: Bulao wordmark, "Booking Confirmed" header, large booking_id, service details, provider name + rating, scheduled time, FULL price line-item breakdown from accepted_quote, a QR code encoding the booking_id, polite Urdu+English thank you. Match the design tokens (gold #b8860b, ink #1a1d29, warm white).
 - e) Uploads the PDF to Cloud Storage at gs://bulao-hackathon-receipts/{booking_id}.pdf and returns the public URL.
 - f) Returns the booking record including receipt_url.
 2. Function generate_confirmation_message(booking: Booking) -> dict that uses Gemini Flash to write a 3-4 line WhatsApp-style confirmation in Urdu and English, addressed to the customer, referencing the provider by name, scheduled time, and quoted price range. Tone: warm, brief, confident.
 3. Wire into the orchestrator. /book accepts {provider_id, accepted_quote_id, user_info} and returns the booking record. The Booking Agent does NOT run during /orchestrate (which is the pre-confirmation pipeline) — it only runs on /book.
 4. Mobile integration: ConfirmationScreen displays the receipt inline (PDF viewer or rendered image). Shows the confirmation message as a styled WhatsApp-style chat bubble UI (green sender bubble, timestamp, double-tick read receipt animation).
 5. The receipt PDF design is critical. Generate one, render as image, and compare side-by-side with three reference designs (Stripe receipts, Apple Wallet passes). It must hold its own visually. Iterate.
- Add tests verifying: booking_id format, Firestore write succeeds, PDF renders without errors, public URL is reachable, quote expiry check works.

Afternoon · the agent-card UI polish

While the backend team finalizes Booking, the Mobile teammate polishes the AgentCard animation. This UI is the agentic moneyshot — the moment the demo wins or loses points on the 20% Agentic Reasoning rubric. Specifications:

AgentCard visual states

Pending: 40% opacity, no border, agent name in muted grey, no content. Card sits in the stack waiting its turn. **Active:** 100% opacity, 2px gold left rule that animates with a subtle pulse (2-second loop), agent name in ink, a small "thinking" Lottie animation in the upper right. Content area shows a typewriter effect as the response streams in (use Flutter's `animated_text_kit` with the `TypewriterAnimatedText` widget). **Done:** 100% opacity, no left rule, a gold checkmark appears in the upper right with a fade-and-scale animation (300ms). Card settles into rest state.

Timing of the demo pipeline

Real LLM latency is variable. To make the demo cinematic, the mobile app should *buffer* agent outputs and reveal them on a deliberate cadence: Intent at +1.5s, Discovery at +3.0s, Ranking at +5.0s. If a real call returns faster, hold the reveal until the cadence; if slower, show a slightly longer thinking animation. This is not deception — it's pacing. Judges read this as professional polish.

PROMPT · POLISH THE AGENT-CARD UI IN FLUTTER

Refine the AgentCard widget at `mobile/lib/widgets/agent_card.dart` and the ProcessingScreen at `mobile/lib/screens/processing_screen.dart`.

1. AgentCard visual states (per the playbook spec):

- Pending: opacity 0.4, no border, agentName in #718096, content hidden.
- Active: opacity 1.0, 2px gold (#b8860b) LeftBorder with a Tween animation that pulses opacity from 0.6 to 1.0 over 2 seconds, infinitely. agentName in #1a1d29. A small Lottie animation in the upper right corner using `lottie-thinking.json` (download a clean monochrome "AI thinking" Lottie from lottiefiles.com; if unavailable, use a `CircularProgressIndicator` with golden color).
- Done: opacity 1.0, LeftBorder gone, a checkmark icon (`Icons.check_circle`, gold) in the upper right appearing with a fade-in + scale-up over 300ms (use `flutter_animate`).

2. ProcessingScreen stacks 6 AgentCards vertically. As the backend response streams in, each card transitions pending → active → done on a forced cadence:

- Intent: revealed and goes Active at $t=1.5s$ from the screen open.
- Discovery: $t=3.0s$.
- Ranking: $t=4.5s$.
- Pricing: $t=6.0s$.
- Booking: $t=8.0s$ (after user taps "Confirm at PKR 2,500").
- Follow-up: $t=10.0s$.

Each transition takes 400ms. If the real backend hasn't returned by the scheduled time, the card stays in active state with the Lottie animation until data arrives.

3. The active-card content uses a `TypewriterAnimatedText` effect on the agent's primary output line. Speed: 30ms per character. For the Pricing Card, type out the estimated total then expand the line-item breakdown.

4. Add a subtle haptic feedback (`HapticFeedback.lightImpact`) when each card transitions to done.

5. After all 6 cards are done, navigate to ConfirmationScreen with a hero animation: the receipt slides up from the bottom with a spring curve.

6. Take a screen recording of the full sequence (use the iOS simulator's built-in recording or scrcpy for Android). Save to docs/antigravity-trace/day3/agent_card_sequence.mp4 — this is one of the most important demo assets.

Make the animations crisp, not flashy. Premium consumer-app energy. Reference: the way iMessage reveals new messages with a subtle scale-in, not the way default Material animations feel.

End-of-day verification

- Ranking Agent produces six-factor reasoning with at least 3 specific numbers per candidate, in both Urdu and English.
- Pricing Agent produces a transparent line-item breakdown with Urdu + English labels for every line, and a sensible estimated range.
- Booking Agent writes to Firestore (with full lifecycle fields) and generates a PDF receipt that looks designed, not generated.
- Mobile app shows 6 agent cards animating in sequence on a real device.
- Pricing Card renders the line-item breakdown clearly before the Confirm button is enabled.
- WhatsApp-style confirmation UI feels indistinguishable from real WhatsApp.
- End-to-end demo flow works: voice/text in → 90 seconds → receipt out.

At end of Day 3, do your first full demo rehearsal on the demo phone. Record it. Watch it back. Note every awkward beat. Day 4 is for fixing those plus the service-quality loop and the dispute path.

07 · DAY 4

Service-quality loop, dispute workflow, edge cases.

Day 4 used to be buffer day. In v2 it has serious work: the Booking Agent gets the service-quality lifecycle (en-route, in-progress, completed with checklist), and the Follow-up Agent absorbs the dispute & escalation workflow that the rubric weights at 15%. Then edge cases, voice input as a stretch goal, and three full dress rehearsals before midnight.

Morning · the service-quality lifecycle

The Booking Agent already wrote the lifecycle fields on Day 3. Today the Mobile teammate brings them to life on screen: en-route progress, arrival confirmation, service-in-progress indicator, and the completion checklist with optional photo evidence. The judge sees this as the agent shepherding a booking through to fair completion — which is exactly what the rubric calls for.

PROMPT · ADD THE SERVICE-QUALITY LIFECYCLE UI

Extend the Bulao mobile app with the post-booking service-quality lifecycle. Backend lifecycle fields already exist on the booking record.

1. Add a new screen `ServiceProgressScreen` at `/service/{booking_id}`. It shows:
 - Provider name and photo placeholder at top.
 - A vertical stepper with five states: confirmed (✓), `en_route`, arrived, `in_progress`, completed. Each step has a timestamp once reached.
 - A live ETA card during `en_route` ("Ali is 22 minutes away") with a fake map showing a moving pin (use Flutter's `google_maps_flutter` with an animated marker).
 - When state = `in_progress`, show an "Estimated time remaining" countdown.
 - When state = completed, show the completion checklist (see step 3).
2. Add backend endpoints:
 - `POST /booking/{booking_id}/transition` with body `{"to": "en_route" | "arrived" | "in_progress" | "completed"}`. Validates state machine transitions. Writes lifecycle timestamp to Firestore. Returns updated booking.
 - `GET /booking/{booking_id}` returns the current booking with lifecycle state.

For the demo, the mobile app has a hidden "next state" button (long-press the provider name 3 times) that advances the state — this lets you cycle through the whole lifecycle in 30 seconds during a demo.
3. Completion checklist UI — when state = completed, show:
 - A checklist with 4 standard items: "Problem diagnosed correctly", "Service completed as agreed", "Work area left clean", "Final price matches estimate". Each item has yes/no toggles.
 - A photo upload placeholder ("Add photo of completed work — optional"). Tapping it shows a camera UI; for the demo, accept any photo and store its base64 in Firestore.
 - A 1-5 star rating input.
 - A free-text "tell us more" field (optional).
 - A "Submit feedback" button that calls `POST /booking/{booking_id}/complete` with the checklist + rating + photo.
4. The completion submission triggers the Follow-up & Dispute Agent: if rating ≥ 3 , normal check-in path; if rating < 3 OR any checklist item is "no", the dispute flow is triggered.

5. Take a screen recording of the full lifecycle (confirmed → en_route → arrived → in_progress → completed → rating submitted) on the demo phone. Save to docs/antigravity-trace/day4/service_lifecycle.mp4.

Midday · the Follow-up & Dispute Agent

The sixth and final agent has two modes. **Reminder mode** handles the pre-service reminder and post-service check-in — the v1 behavior. **Dispute mode** is new in v2: when the user rates < 3 stars or taps "Report an issue", the agent classifies the dispute, proposes a fair resolution, and either auto-applies it or escalates to a human. This is 15% of the rubric on its own.

PROMPT · BUILD THE FOLLOW-UP & DISPUTE AGENT

Implement the Follow-up & Dispute Agent at backend/agents/followup_agent.py. It has two modes; both use Gemini Flash for natural-language message generation but the decision logic is mostly deterministic Python.

== REMINDER MODE ==

- Function `schedule_followup(booking: Booking) -> FollowupPlan` that:
 - Computes a reminder time = `booking.scheduled_time - 30 minutes`.
 - Computes a check-in time = `booking.scheduled_time + 2 hours`.
 - Creates two Cloud Scheduler jobs in `asia-south1`, both POSTing to `/webhooks/followup` with payloads `{"booking_id": ..., "type": "reminder"}` and `{"type": "checkin"}`.
 - Writes the followup plan to Firestore at `bookings/{booking_id}/followups/`.
 - Returns the `FollowupPlan`.
- POST `/webhooks/followup` webhook generates the message (using the Follow-up Agent system prompt, Appendix A.6) and writes it to Firestore. For the demo, the message renders as a fake push notification in the Flutter app.

== DISPUTE MODE ==

- Function `handle_dispute(booking_id: str, rating: Optional[int], checklist_failures: List[str], user_message: Optional[str]) -> DisputeResolution`.
- Classify the dispute type:
 - "no_show" — checklist contains "provider didn't arrive"
 - "quality_complaint" — rating <= 2 OR checklist has "Service completed as agreed" = no
 - "price_disagreement" — checklist has "Final price matches estimate" = no
 - "overrun" — actual completion time > estimated * 1.5
 - "other" — fallback
- Severity classification (low / medium / high):
 - high: no_show OR provider has 2+ existing strikes
 - medium: quality_complaint with rating 1 OR price disagreement > 30% over quote
 - low: everything else
- Resolution rules (deterministic):
 - no_show → full_refund + provider_warning + compensation (PKR 300)
 - quality_complaint, low severity → partial_refund (50%) OR free re_service (user choice)
 - quality_complaint, high severity → full_refund + provider_warning
 - price_disagreement → reconcile against accepted_quote; if provider charged > quote range, refund the overage; if user just disagrees with quote, no refund (they accepted upfront) but offer 10% loyalty credit on next booking
 - overrun → partial_refund of overrun hours
 - if provider.strikes after this becomes 3 → blacklist + human_escalation
- Generate `resolution_message` in Urdu + English using Gemini Flash. The message must: acknowledge the issue empathetically, state the resolution clearly, give a clear next-step or contact path, and end with a thank-you. NOT defensive. NOT bureaucratic.

8. Write the resolution to Firestore at bookings/{booking_id}/disputes/{dispute_id} with all fields.
9. Mobile integration: after the user submits the completion checklist with a low rating, the app navigates to DisputeScreen which shows: a) the user's complaint summary, b) the agent's classified resolution, c) accept/decline buttons (declining triggers human_escalation: "Hum aapse 24 ghante mein contact karenge"), d) for accepted resolutions, a confirmation message and an estimated refund timeline.
10. Add endpoint GET /booking/{booking_id}/dispute that returns the current dispute state.
11. Tests at backend/tests/test_followup_dispute.py covering both modes: a) reminders are scheduled correctly, b) dispute classification is correct for each scenario, c) resolutions match the deterministic rules, d) strikes increment correctly, e) blacklist triggers at 3 strikes, f) urdu and english resolution messages are both populated.

Late morning · edge cases that kill demos

Spend three hours on the failure modes a judge will probe at finals. Each is one paragraph of fallback dialogue in the agent prompt plus one fallback screen in Flutter:

Vague location	User says "mere ghar pe". Intent Agent returns location: null with confidence < 0.7 and clarification_question = "Aap kis ilaake mein hain?". Mobile app shows the question as a tappable chip; user picks from known neighborhoods.
Zero providers	Discovery returns no_match_reason. Mobile shows a friendly Urdu fallback: "Filhal koi provider available nahi hai is waqt. Hum aapko notify karenge jab koi mile." Logs the intent to Firestore for later matching.
Quote expired	User takes > 15 min to confirm. /book returns quote_expired. Mobile shows: "Quote expired ho gaya, naya estimate le rahe hain..." and re-runs the pipeline. New quote shows side-by-side with old for transparency.
Service unclear	Intent Agent confidence between 0.5 and 0.7. Returns the best guess with a clarification question. Mobile app shows the intent with the question chip below.
User changes mind on price	After Pricing Card, user taps "too expensive". App shows the lowest-price alternate from Discovery with a fresh quote (lower because of cheaper provider rate).
Provider cancels post-confirmation	Backend webhook simulates a provider cancellation. The Follow-up Agent fires an auto-reschedule: Discovery re-runs to find a substitute, mobile gets a notification with the new provider + new quote, user confirms or rejects.
Disputed price after service	User completes service and disputes the price in the checklist. Dispute Agent classifies as price_disagreement, reconciles against accepted_quote, proposes appropriate refund.
Network failure	Backend unreachable. Mobile shows a polite offline state with retry. Don't show raw error codes.

Afternoon · voice input (stretch goal)

Voice is the difference between a good demo and a great one, but a flaky voice demo is worse than no voice demo. Decision point at 4pm: if voice works reliably on three out of three test runs, keep it. If not, ship text-only.

PROMPT · ADD VOICE INPUT VIA GEMINI 3 PRO AUDIO

Add voice input to the Bulao mobile app and backend.

1. Mobile: the HomeScreen mic button uses the record package to capture audio when held. On release, the audio is encoded as base64 and sent to a new backend endpoint POST /transcribe.
 2. Backend: implement /transcribe endpoint at backend/main.py. It accepts {"audio_base64": str, "mime_type": str}. Decodes the audio. Sends it to Gemini 3 Pro with a system prompt: "Transcribe this audio. The speaker may use Urdu, Roman Urdu, or English. Return ONLY the transcribed text, no preamble." Returns {"text": str, "detected_language": str}.
 3. Mobile flow on release of the mic button:
 - a) Show a loading shimmer on the home screen with the message "Sun raha hoon..."
 - b) Call /transcribe with the audio.
 - c) When transcription returns, display the transcribed text briefly (1 second) so the user knows they were heard correctly.
 - d) Then navigate to ProcessingScreen and call /orchestrate with the transcribed text.
 4. Edge cases:
 - Audio shorter than 1 second: don't send, show "Thora aur boliye" (Speak a bit more).
 - Transcription returns empty: show fallback "Hum samajh nahi paaye. Wapas try karein ya likh ke bhejein."
 - Network slow: show shimmer for max 4 seconds, then fall back to a text input.
 5. Test on a real Android device with three different voices (each teammate records the same Urdu sentence and verifies the transcription). If accuracy is below 80%, fall back to text-only for the demo.
- Important: NEVER send raw audio to Gemini without checking duration first; very short audio causes hangs.

Evening · three full dress rehearsals

Between 7pm and 10pm, run the complete demo three times. Use the actual demo phone. Time every run with a stopwatch — the target is 3 minutes 30 seconds total. After each run, the Product/Urdu lead identifies the three weakest beats and the team fixes them before the next run.

What to time

- Time from app launch to mic button press: under 5 seconds.
- Time from mic release to Intent card appearing: 1.5–2.5 seconds.
- Time from Intent card to Ranking card with reasoning: 3.5–5 seconds.
- Time from confirm to receipt rendering: 1.5–2 seconds.
- Total demo time: 3:00–3:30 inclusive of voiceover.

Sleep early tonight. Day 5 is editing, documentation, submission — tasks that punish tired brains. The demo video is the single highest-leverage thing you'll make this week.

08 · DAY 5 (MAY 20)

Demo video, documentation, submit.

Today the team becomes a media production house and a documentation factory. The creative lead directs the demo. The developers write the README and assemble the Antigravity trace package. Submission goes in by 6pm — not midnight. The submission form goes live after Selection Day; check the email for the link.

Morning · the demo video (4 hours)

The shot list

Shoot to this storyboard. Total runtime target: **4:30** (under the 5-minute limit; v2 adds the Pricing and Dispute beats). Voiceover in Urdu with English subtitles. Music: subtle, slow-tempo, no lyrics — from Epidemic Sound or Artlist.

0:00–0:20	Open on the demo phone unlocking. App icon taps open. Home screen appears. Voiceover: "Pakistan mein ghar ke kaam ke liye plumber ya electrician dhoondhna ek puraana masla hai." (Subtitle: A perennial problem in Pakistan.)
0:20–0:40	Camera pushes in on the press-and-hold button. User presses and holds, speaks in Urdu ("AC theek karwana hai G-13 mein, inverter wala hai"). Transcribed text appears for a beat. Voiceover: "Aap bolte hain. Bulao samajhta hai."
0:40–1:00	Intent Agent card activates. Reveals: service, location, time, urgency, complexity ("intermediate"), confidence (0.92). Voiceover: "Pehla agent samajhta hai kya kaam hai, kab chahiye, kitna complex hai."
1:00–1:30	Discovery Agent card activates. Map slides in. 7 provider pins. A small badge: "11am slot busy — alternates available at 11:30." Voiceover: "Doosra agent providers dhoondhta hai aur scheduling conflicts check karta hai."
1:30–2:00	Ranking Agent card activates. The reasoning types out: "Ali sab se behtar — 1.4km, 4.8 rating, 98% on-time, inverter AC specialist." Tap "Why?" reveals 6-factor bar chart. Voiceover: "Teesra agent chhe factors ke saath rank karta hai."
2:00–2:30	NEW · Pricing Agent card activates. Line-item breakdown reveals: visit fee, service estimate, complexity (intermediate, 1.25×), distance, urgency, surge, welcome discount. Estimated range "PKR 2,260–2,890". Voiceover: "Chautha agent transparent pricing dikhata hai. Surprise nahi."
2:30–2:50	User taps Confirm. Booking Agent card activates. Receipt slides up. Camera lingers 2 seconds on receipt showing booking_id, full line-items, QR code. WhatsApp-style confirmation chat bubble appears.
2:50–3:20	Lifecycle accelerated: provider "en route" pin animates across mini-map, arrives, status changes to "in progress", then "completed". Completion checklist appears with toggles. Voiceover: "Service track hota hai."

3:20–3:50	NEW · User taps 2 stars (showing a quality issue). DisputeScreen appears. The Follow-up Agent classifies as "quality_complaint", proposes "partial refund PKR 1,250 OR free re-service". User taps accept refund. Confirmation message: "PKR 1,250 wapas 3-5 din mein." Voiceover: "Aakhri agent dispute ko insaaf ke saath handle karta hai."
3:50–4:10	Cut to clean reminder fired in a happy parallel flow (no dispute) for closing balance. Notification banner slides in: "Ali aapke ghar 30 minute mein."
4:10–4:30	Closing card: Bulao logo, 6-agent flow diagram, team credits. Final voiceover: "Bulao — bolo, aur kaam ho jaye."

Production notes

- Shoot on the real demo phone. Use a tripod. Light the phone with a soft side light to avoid glare. Multiple takes per shot — aim for at least 3 clean takes of each.
- Record voiceover separately in a quiet room with a phone or USB mic. Don't rely on the phone's built-in mic for voice.
- Edit in DaVinci Resolve (free) or CapCut. Apply a subtle film grain and warm tone — premium consumer brand feel, not corporate.
- Add subtitles in both Urdu and English. Burn them in — don't rely on YouTube captions.
- End card: "Bulao — Team [name] — AI Seekho 2026" with the four teammate names.
- Export at 1080p, mp4, H.264. Upload to YouTube as unlisted. Include the URL in the README.

Afternoon · documentation (2 hours)

Three documents need to ship: the README, the Antigravity trace package, and a brief architecture document.

README structure

FINAL README SECTIONS (TOP-LEVEL)

# Bulao – Bolo, aur kaam ho jaye	
1. Project Overview	– what + why, 2 paragraphs
2. Demo Video	– YouTube unlisted URL, large embed
3. The 5-Agent Architecture	– diagram + agent descriptions
4. How Antigravity Was Used	– 4 screenshots, 200 words each
5. Tools & APIs	– bulleted list
6. Setup & Running Locally	– step-by-step terminal commands
7. Deployment	– gcloud commands
8. The 50-Example Urdu Dataset	– link + summary
9. Roadmap / What We'd Build Next	– 5 bullets
10. Team	– names, roles, photos
11. Acknowledgements	– InnoVista, MoITT, Telenor, Google

Antigravity trace package

Create docs/antigravity-trace/FINAL.pdf — a single PDF that contains the highlight reel of screenshots from the build week. Eight to twelve images max, with one-line captions. The order: a Manager View with 3+ parallel agents (Day 1), a Plan Artifact for the Intent Agent (Day 2), a browser recording still showing the agent testing the Flutter app (Day 2), a knowledge-base screenshot showing saved context (Day 3), a diff with reviewer comments (Day 3), and a final Manager View showing the polish/edge-case work (Day 4). This PDF goes into the submission package as the Agent Trace deliverable.

Late afternoon · submit (1 hour)

- Final APK signed and uploaded to a public link (Drive, GitHub releases).
- Demo video uploaded to YouTube as unlisted, URL in README.
- Antigravity Trace PDF generated and uploaded.

- README finalized and pushed to main branch.
- GitHub repo set to public (or invite judges as collaborators — check the form).
- Backend Cloud Run service is live and healthy.
- Form submission completed — double-check all fields.
- Screenshot the submission confirmation page. Save it.

After you submit, do not touch the code. Resist the urge to push one more fix. Sleep. Tomorrow you start preparing for Regional Finals — a different kind of work.

APPENDIX A

The five agent system prompts.

These are the actual system prompts to paste into `backend/agents/prompts/`. They are the source of truth for the agents' behavior. Iterate on these — not on Python code — when an agent misbehaves. Commit each version with a git message describing what you changed and why.

A.1 · Intent Agent

Model: **gemini-flash-latest**. Temperature: **0.1**. File: `backend/agents/prompts/intent_agent_system.md`.

PROMPT

You are the Intent Agent for Bulao, a voice-first AI booking platform for home services in Pakistan. Your job is to extract structured booking information from natural-language requests.

The user may speak in Urdu, Roman Urdu (Urdu written in Latin script), English, or any code-switched mix. You must handle all of these naturally.

EXTRACT THESE FIELDS

service_type – one of:
 plumber, electrician, ac_technician, geyser_technician, carpenter, painter, beautician, tutor, appliance_repair, gas_leak_specialist

location – the neighborhood or area (e.g. "G-13", "F-10", "Bahria Town Phase 4", "DHA Karachi").
 If the user says only "mere ghar pe" or "at home", return null and let the app prompt for location.

city – inferred from neighborhood; default "Islamabad" if ambiguous.

time_window – when they need service. Use one of these tokens, or an ISO datetime if explicit:
 now, today_morning, today_afternoon, today_evening, today_night, tomorrow_morning, tomorrow_afternoon, tomorrow_evening, this_friday, this_weekend, next_week, flexible

urgency – emergency | high | normal | flexible.
 emergency = within 1 hour, water/gas leak, no power
 high = same day
 normal = within a few days
 flexible = whenever

gender_preference – male | female | any.
 Default "any". Auto-detect "female" for women-specific services like bridal beauticians or if the user explicitly asks for a female provider.

budget_range – extracted only if explicit (e.g. "5000 se 10000 ke beech"). Otherwise null.

raw_notes – any additional context the user mentioned that doesn't fit other fields (e.g. "geyser leak", "AC chal raha hai par thanda nahi", "bridal makeup").

NEW IN v2 – JOB COMPLEXITY CLASSIFICATION:

job_complexity – "basic" | "intermediate" | "complex"
 basic = any provider in the category can do it (unclog drain, change tap washer, swap a switch, basic cleaning, simple haircut)
 intermediate = needs experience (AC service, geyser repair, electrical fault diagnosis, ceiling fan install, full makeup)
 complex = needs specialization, certification, or tools (inverter AC repair, gas line work, rewiring, structural plumbing, bridal/HD makeup)

Use the raw_notes and service description to decide. Default to "intermediate" if unsure.

NEW IN v2 – CONFIDENCE SCORING:

confidence – float 0.0 to 1.0 indicating how sure you are of your interpretation.
 1.0 = explicit, unambiguous request with all key fields clear
 0.7-0.99 = clear with minor ambiguity (e.g. specific service but vague time)
 0.5-0.69 = significant ambiguity (unclear location, multiple plausible services)
 <0.5 = too vague to act on confidently

clarification_question – ONLY populate if confidence < 0.7. A natural one-sentence question to ask the user. Format: { "urdu": "...", "english": "..."}
 Otherwise null. The question should target the MOST ambiguous field.

URDU / ROMAN URDU CHEAT-SHEET

"Mujhe ___ chahiye"	= I need ___
"___ bhejo / bhejein"	= send ___
"___ wala / wali"	= ___ guy / lady (e.g. bijli wala = electrician)
"Theek karwana"	= to repair
"Kal subah / sham"	= tomorrow morning / evening
"Abhi / Foran"	= now / immediately
"Jaldi se"	= quickly
"Kabhi bhi"	= whenever (= flexible)
"Ghar pe"	= at home (need actual location)
"Bijli wala"	= electrician
"Pani wala / plumber"	= plumber
"AC wala / AC tech"	= ac_technician
"Geyser wala"	= geyser_technician

EDGE CASES

- Ambiguous service: pick most likely, add note in raw_notes, reduce confidence to 0.5-0.69.
- Multiple services: extract the primary, mention others in raw_notes.
- No location: location = null, confidence drops to 0.5, clarification_question asks for it.
- Vague time: "kabhi bhi" -> flexible, "jaldi" -> high.

OUTPUT FORMAT – JSON ONLY. No preamble, no explanation, no markdown fences:

```
{
  "service_type": "ac_technician",
  "location": "G-13",
  "city": "Islamabad",
  "time_window": "tomorrow_morning",
  "urgency": "normal",
  "gender_preference": "any",
  "budget_range": null,
  "raw_notes": "AC chal raha hai par thanda nahi",
  "job_complexity": "intermediate",
  "confidence": 0.92,
  "clarification_question": null
}
```

LOW-CONFIDENCE EXAMPLE (vague location):

```
{
  "service_type": "plumber",
  "location": null,
  "city": "Islamabad",
  "time_window": "now",
  "urgency": "high",
  "gender_preference": "any",
  "budget_range": null,
  "raw_notes": "pani leak ho raha hai",
  "job_complexity": "basic",
  "confidence": 0.55,
  "clarification_question": {
    "urdu": "Aap kis ilaake mein hain? (jaise G-13, F-10)",
    "english": "Which neighborhood are you in? (e.g. G-13, F-10)"
  }
}
```

OUTPUT NOTHING EXCEPT THIS JSON.

A.2 · Discovery Agent (no LLM)

The Discovery Agent is plain Python — no LLM. The behavior contract is documented below for the developer implementing it.

PROMPT

The Discovery Agent does NOT use an LLM. It is plain Python – a filter and scorer over the provider database. This contract document defines its behavior.

INPUT

- intent: validated Intent object
- user_location: optional LatLng

ALGORITHM

1. Filter providers where intent.service_type is in provider.service_categories.
2. If intent.location is set, filter to providers in that neighborhood OR within 5km of the geocoded neighborhood center.
3. If intent.gender_preference is "female" or "male" (not "any"), filter accordingly.
4. Compute distance_km using haversine from user_location, or from the neighborhood center if user_location is None.
5. Compute availability_status by comparing provider.available_slots to intent.time_window:
 - "available_now": next slot within 60 minutes
 - "next_slot_within_2h": next slot within 2 hours
 - "next_slot_today": next slot within 12 hours
 - "tomorrow_or_later": otherwise
6. Sort by (distance_km asc, availability_status priority, rating desc).
7. Return top 8 candidates.

OUTPUT – List[ProviderCandidate]:

```
{ id, name, service_categories, distance_km, neighborhood, rating,
  completed_jobs_in_area (filtered to providers' completed jobs in this neighborhood),
  availability_status, next_slot, gender, years_experience, phone_masked }
```

PHONE MASKING

Only last 4 digits visible (e.g. "+923XX-XXX1234" → "+92 XXX-XXX-1234"). Full phone revealed only after booking confirmation.

A.3 · Ranking Agent

Model: **gemini-3-pro-latest**. Temperature: **0.4**. File: backend/agents/prompts/ranking_agent_system.md.

PROMPT

You are the Ranking Agent for Bulao. You receive a structured user intent and up to 8 service-provider candidates, and you produce a ranked recommendation with clear reasoning in both Urdu and English.

INPUT

intent – the validated Intent object from the Intent Agent.

candidates – a list of up to 8 ProviderCandidate objects, each with:

```
{ id, name, service_categories, distance_km, neighborhood, rating,
  completed_jobs_in_area, availability_status, next_slot, gender,
  years_experience, phone_masked }
```

YOUR JOB

1. Score each candidate against the user's intent. Weights vary by urgency:
 - emergency: distance 40%, availability 40%, rating 20%
 - high: distance 30%, availability 30%, rating 25%, completed_jobs_in_area 15%
 - normal: rating 35%, completed_jobs_in_area 25%, distance 25%, availability 15%
 - flexible: rating 40%, completed_jobs_in_area 30%, price (if available) 20%, distance 10%
2. Pick the single highest-scoring candidate as recommended_id.
3. Identify the top three (by total score) as top_three_ids.
4. Write reasoning in English (1-2 sentences). MUST reference specific numbers from the chosen candidate – distance in km, rating, completed_jobs_in_area, availability status. No generic phrases like "best for you" or "highly recommended". Be concrete.
5. Write the same reasoning in natural Urdu (Roman script is fine). It must sound like a Pakistani speaker, not translated English. Use natural phrases like "sirf X km door", "aapke ilaake mein Y jobs ki Z rating", "abhi available hai".
6. Honestly state the tradeoffs with the second and third options if the second-place candidate has a meaningful edge on any dimension (lower price, closer distance, higher rating, gender match).
7. Confidence – "high" if the top candidate is meaningfully better than the second, "medium" if it's a close call, "low" if no candidate is a great match.

OUTPUT FORMAT — JSON ONLY

```
{
  "recommended_id": "prov_042",
  "top_three_ids": ["prov_042", "prov_017", "prov_089"],
  "reasoning_english": "Ali Plumbing is the strongest match – only 1.4km away, 4.8 rating from 47 completed jobs in G-13, available within 30 minutes.",
  "reasoning_urdu": "Ali Plumbing sab se behtar hai – sirf 1.4 km door, G-13 mein 47 jobs ki 4.8 rating, aur 30 minute mein available.",
  "tradeoffs": "If you prefer a lower price, Rashid Plumbing is 15% cheaper but 30 minutes further and rated 4.5.",
  "confidence": "high"
}
```

CONSTRAINTS

- recommended_id MUST be one of the candidate IDs.
- Each reasoning string MUST contain at least one digit (a specific number).
- Urdu reasoning MUST be natural – avoid literal translation.
- If candidates list is empty, return: { "recommended_id": null, "error": "no_candidates" }.

OUTPUT NOTHING EXCEPT THE JSON.

A.4 · Pricing Agent (new in v2)

Model: **gemini-flash-latest**. Temperature: **0.2**. File: `backend/agents/prompts/pricing_agent_system.md`. The pricing math is deterministic Python; this prompt only writes the natural-language explanation that goes with the line items.

PROMPT

You are the Pricing Agent for Bulao. You produce a transparent, line-item price quote for a service booking. The user sees this quote BEFORE confirming, so it must be honest, complete, and feel fair to both customer and provider. Transparency is the product, not a feature.

INPUT (structured, not free text):

- intent: validated Intent (urgency, location, job_complexity, time_window)
- provider: recommended ProviderCandidate (base_visit_fee_pkr, rate_per_hour_pkr, distance_km)
- market_demand: float 0.0-1.0 (0.5 is normal; >0.7 triggers surge)
- is_first_booking: boolean (true → loyalty_discount applies)
- line_items: list of computed line items (the Python code did the math; you write the words)
- estimated_total_pkr: int
- estimated_range_pkr: (low, high) tuple

THE LINE ITEMS YOU'LL EXPLAIN (always in this order, even if zero):

1. base_visit_fee – provider's flat visit fee
2. distance_fee – beyond first 3km, PKR 50 per km
3. estimated_service_fee – rate_per_hour × estimated_hours (depends on complexity)
4. complexity_multiplier – ×1.0 (basic), ×1.25 (intermediate), ×1.6 (complex)
5. urgency_surcharge – emergency +500, high +200, normal 0, flexible -100
6. surge_factor – applied if market_demand > 0.5
7. loyalty_discount – -50 to -200 PKR if is_first_booking is true

YOUR JOB

Write a natural-language explanation that contextualizes the line items in both Urdu and English. 1-2 sentences each. Mention surge ONLY if surge_factor > 1.0. Mention complexity multiplier ONLY if not "basic". Mention loyalty_discount ONLY if is_first_booking.

OUTPUT FORMAT — JSON ONLY

```
{
  "explanation_english": "Your AC repair estimate is PKR 2,260-2,890. This includes the visit fee, an intermediate complexity adjustment for inverter ACs, and a small surge because demand is high today. Welcome discount applied.",
  "explanation_urdu": "Aapka AC repair ka estimate PKR 2,260-2,890 hai. Is mein visit fee, inverter AC ke liye intermediate complexity adjustment, aur thora surge shamil hai kyunke aaj demand zyada hai. Welcome discount apply ho gaya.",
  "fairness_note": "Provider receives PKR 1,920 after platform fee."
}
```

RULES

- DO NOT hide unfavorable line items. Surge and complexity multipliers must be named.
- Urdu must be natural – not literal translation of the English.
- Mention the price range, not just the point estimate.
- Always include the fairness_note so the user knows the provider isn't being underpaid.

OUTPUT NOTHING EXCEPT THE JSON.

A.5 · Booking Agent

Model: **gemini-flash-latest**. Temperature: **0.5**. File: backend/agents/prompts/booking_agent_system.md.
The Firestore write and PDF generation are plain Python; only the message generation uses an LLM.

PROMPT

You are the Booking Agent – specifically, the message-generation sub-task within it. The rest of the Booking Agent is plain Python (Firestore writes, PDF generation, booking ID creation). Your job is to write the confirmation message that gets shown to the user after they confirm a booking.

INPUT (passed as a structured payload, not free text):

- booking: { booking_id, service_type, scheduled_time, location, expected_price_range }
- provider: { name, rating, years_experience }
- user_name: optional

YOUR JOB

Write a 3-4 line WhatsApp-style confirmation message in BOTH Urdu and English. Tone: warm, brief, confident – like a thoughtful assistant. NOT corporate. NOT excessive.

THE MESSAGE MUST

- Greet the user (use first name if provided, otherwise neutral).
- Confirm the booking with the booking_id.
- Mention the provider by name and one positive attribute (rating or experience).
- State the scheduled time naturally ("kal subah 10 baje" rather than "tomorrow 10:00 AM").
- End with a friendly close.

EXAMPLE OUTPUT

```
{
  "english": "Hi Ayesha, your plumbing booking is confirmed. Ali (4.8 rating, 12 years experience) will reach G-13 tomorrow morning at 10. Booking ID: BUL-20260516-9KP3X4. We'll send a reminder 30 minutes before. – Bulao",
  "urdu": "Assalam-o-Alaikum Ayesha, aapka plumbing booking confirm ho gaya. Ali (4.8 rating, 12 saal ka tajurba) kal subah 10 baje G-13 pohanch jayenge. Booking ID: BUL-20260516-9KP3X4. 30 minute pehle reminder bhej dein ge. – Bulao"
}
```

OUTPUT FORMAT – JSON ONLY with keys "english" and "urdu". No preamble. No markdown.

A.6 · Follow-up & Dispute Agent

Model: **gemini-flash-latest.** Temperature: **0.4.** File: backend/agents/prompts/followup_dispute_agent_system.md. Has two modes: the happy-path reminder/check-in flow, and the dispute classification + resolution flow when something goes wrong. Both share the same prompt file.

PROMPT

You are the Follow-up & Dispute Agent's message generator. The agent itself is a Python orchestrator + Cloud Scheduler; you write the words. The agent has TWO MODES:

MODE 1 – REMINDER / CHECK-IN (normal path):

Triggered at: 30 minutes before scheduled service (reminder), or 2 hours after completion (check-in).

MODE 2 – DISPUTE (triggered when rating < 3 OR user taps "Report an issue"):

Triggered when the completed-service feedback shows a problem.

```
=====
MODE 1: REMINDER + CHECK-IN
=====
```

INPUT

- booking: full booking record
- provider: provider details
- type: "reminder" | "checkin"

REMINDER MESSAGE – 2-3 lines, Urdu + English. Tone: helpful, friendly, not pushy. Include polite heads-up that the service is in 30 minutes, provider name and ETA, and a way to message the provider if needed.

CHECK-IN MESSAGE – 2-3 lines, Urdu + English. Tone: warm, brief. Check on how the service went, prompt for a 1-tap rating (1-5 stars), thank you.

OUTPUT FORMAT — JSON ONLY

```
{
  "mode": "reminder",
  "english": "...",
  "urdu": "...",
  "cta": "rate" | "contact_provider" | "none"
}
```

```
=====
MODE 2: DISPUTE CLASSIFICATION + RESOLUTION
=====
```

INPUT

- booking: completed booking record (includes accepted_quote for reference)
- user_complaint: free-text user complaint (in Urdu/Roman Urdu/English)
- service_checklist: list of {item: str, status: "yes"|"no"}
- rating: int 1-5 (if provided)

YOUR JOB

Classify the dispute, propose a resolution, and write the user-facing message.

CLASSIFICATION (pick one):

- no_show – provider did not arrive
- quality_complaint – service was poor / incomplete
- price_disagreement – actual_price diverged from accepted_quote significantly
- overrun – service took much longer than expected
- damage – provider damaged something during service
- other – too complex; needs human review

RESOLUTION (pick one, preferring earlier options when applicable):

- partial_refund – user gets PKR X back; provider compensated from platform reserve
- full_refund – full booking amount returned; provider no compensation
- re_service – provider returns to redo work (no cost or with discount)
- provider_warning – warning issued; rating impact; no money moves
- escalate_to_human – dispute too complex; human reviews within 24h

RESOLUTION RULES

- For "quality_complaint" with rating 2: default partial_refund (50%) OR re_service.
- For "quality_complaint" with rating 1: default full_refund.
- For "no_show": always full_refund + provider_warning.
- For "price_disagreement": refund the difference between accepted_quote and actual_price.
- For "damage": always escalate_to_human (legal/insurance involved).
- For "other": always escalate_to_human.

OUTPUT FORMAT — JSON ONLY

```
{
  "mode": "dispute",
  "classification": "quality_complaint",
  "resolution": "partial_refund",
  "refund_pkr": 1250,
  "message_english": "We're sorry the service didn't meet expectations. We're refunding PKR 1,250 (50% of the service fee). Refund processed in 3-5 days. Ali has been notified for review.",
  "message_urdu": "Maafi mangte hain ke service achi nahi rahi. PKR 1,250 (service fee ka 50%) wapas kar rahe hain. Refund 3-5 din mein process hoga. Ali ko bhi review ke liye notify kar diya hai.",
  "escalate_to_human": false,
  "internal_notes": "Customer reported incomplete work on AC; provider on-time but quality below standard. Auto-resolve approved."
}
```

```
=====
GLOBAL RULES (both modes)
=====
```

- Tone: warm, brief, confident. Never corporate. Never excessive.
- Urdu must be natural Pakistani speech, not literal translation.
- For disputes, ALWAYS acknowledge the user's feeling before stating the resolution.
- Never blame the user. Never make the provider look incompetent in user-facing copy (internal_notes is private).
- If escalate_to_human is true, the user-facing message must reassure them that a human will review within 24 hours.

OUTPUT NOTHING EXCEPT THE JSON.

APPENDIX B

Mock data schemas.

The data structures used across the system. Generate the actual data files using Antigravity on Day 1 by pasting the relevant schema in a prompt and asking the agent to produce 100 realistic Pakistani entries.

B.1 · Service providers (mock database)

```
// backend/data/providers.json – 100 entries total
// Each entry conforms to this schema:

{
  "id": "prov_001",
  "name": "Ali Plumbing Works",
  "service_categories": ["plumber", "geyser_technician"],
  "lat": 33.6584,
  "lng": 73.0479,
  "neighborhood": "G-13",
  "city": "Islamabad",
  "rating": 4.8,
  "completed_jobs": 127,
  "completed_jobs_by_neighborhood": {
    "G-13": 47,
    "G-11": 38,
    "F-11": 22
  },
  "available_slots": [
    "2026-05-15T10:30:00+05:00",
    "2026-05-15T14:00:00+05:00",
    "2026-05-16T09:00:00+05:00"
  ],
  "phone": "+923331234567",
  "gender": "male",
  "years_experience": 12,
  "languages": ["urdu", "english"],
  "price_range_pkr": [1500, 5000],
  "verified": true
}

// Cover at least these neighborhoods with 5+ providers each:
// F-6, F-7, F-8, F-10, F-11, G-7, G-9, G-10, G-11, G-13,
// Bahria Town (Phases 1-8), DHA (Phases 1-7), Saddar Pindi, Pindi Cantt
```

B.2 · Urdu intent examples (eval dataset)

```
// backend/data/intent_examples.jsonl – 50 lines, one JSON per line
// Format:

{"input": "Mujhe G-13 mein kal subah AC theek karwana hai", "expected": {"service_type":
"ac_technician", "location": "G-13", "city": "Islamabad", "time_window": "tomorrow_morning",
"urgency": "normal", "gender_preference": "any", "raw_notes": null}}
```

// Distribution targets:

- // Languages: 20 Urdu/Roman Urdu, 10 English, 20 code-switched
- // Services: each of 10 categories has at least 3 examples
- // Locations: at least 8 distinct neighborhoods
- // Urgency: 8 emergency, 25 normal, 17 flexible
- // Edge cases: 5 ambiguous, 5 with mid-sentence corrections, 5 with multiple services

B.3 · Booking records (Firestore)

```
// Firestore collection: bookings
// Each document:

{
  "booking_id": "BUL-20260516-9KP3X4",
  "user_id": "user_abc123",
  "provider_id": "prov_042",
  "service_type": "plumber",
  "location": "G-13",
  "city": "Islamabad",
  "scheduled_time": "2026-05-17T10:00:00+05:00",
  "urgency": "normal",
  "status": "confirmed", // confirmed | in_progress | completed | cancelled
  "created_at": "2026-05-16T14:30:00+05:00",
  "expected_price_range_pkr": [1500, 5000],
  "actual_price_pkr": null,
  "receipt_url": "https://storage.googleapis.com/bulao-receipts/BUL-...pdf",
  "confirmation_message": {
    "urdu": "...",
    "english": "..."
  },
  "intent_snapshot": { /* full Intent object that produced this booking */ },
  "ranking_snapshot": { /* full Ranking output, for audit */ }
}

// Subcollection: bookings/{booking_id}/followups
{
  "type": "reminder", // reminder | checkin
  "scheduled_for": "2026-05-17T09:30:00+05:00",
  "fired_at": null,
  "message": null,
  "user_response": null
}
```

APPENDIX C

The Antigravity prompt library.

Every Antigravity prompt this team will use, in one place. Sections 04–08 above include the full prompts for each build day; this appendix adds the smaller utility prompts used for debugging, refactoring, and demo prep. Copy-paste freely.

C.1 · Build prompts (recap)

Full text in their respective day sections:

- **Day 1 Prompt A** — Mobile track scaffolding (Section 04, page 4).
- **Day 1 Prompt B** — Backend ADK scaffolding (Section 04, page 5).
- **Day 1 Prompt C** — Infrastructure and trace setup (Section 04, page 6).
- **Day 2** — Build the real Intent Agent (Section 05).
- **Day 2** — Build the Discovery Agent (Section 05).
- **Day 3** — Build the Ranking Agent (Section 06).
- **Day 3** — Build the Booking Agent + Receipt PDF (Section 06).
- **Day 3** — Polish the agent-card UI (Section 06).
- **Day 4** — Build the Follow-up Agent (Section 07).
- **Day 4** — Add voice input via Gemini 3 Pro audio (Section 07).

C.2 · Utility prompts for the week

DEBUG A FAILING TEST

Run the failing test at . Read the assertion error. Identify the root cause. Propose a minimal fix — change one thing, not five. Run the test again to verify. If the test is wrong (not the code), update the test and explain why in the commit message.

REFACTOR FOR READABILITY

Refactor for readability without changing behavior. Specifically: extract magic numbers into named constants at the top of the file, split functions longer than 40 lines, add a docstring to every public function, and ensure imports are sorted (stdlib, third-party, local). After refactoring, run the existing tests and confirm they still pass.

GENERATE THE 100 PROVIDERS.JSON ENTRIES

Generate 100 realistic Pakistani service provider entries for `backend/data/providers.json` conforming to the schema in Appendix B of the Bulao engineering playbook. Distribute providers across at least 14 neighborhoods. Use authentic Pakistani names (mix of male and female). Each provider must have 2-5 available_slots in the next 7 days. Phone numbers in +923XX-XXXXXXX format. Ratings between 3.5 and 5.0 with a realistic distribution (most around 4.2-4.7). Save the file and validate it parses as JSON.

BUILD THE DEMO-MODE HARDCODED PATH

Add a `DEMO_MODE` environment variable. When `DEMO_MODE=true`, the `/orchestrate` endpoint accepts any input but always returns a pre-canned response that's perfect for the demo (the response is loaded from `backend/data/demo_canned_response.json`). This is the safety net for the live demo — if the real LLM call fails or is slow at finals, we flip the flag and the demo still flows. Make sure `DEMO_MODE` is logged loudly so we don't accidentally leave it on in production.

GENERATE THE DEMO VIDEO CLOSING CARD

Create an animated closing card for the Bulao demo video as a 5-second standalone video clip (1080×1920 portrait for phone-shape demos, or 1920×1080 landscape). The clip starts with a black screen, the Bulao wordmark fades in (use Helvetica Bold, color #b8860b on #1a1d29), the tagline "Bolo, aur kaam ho jaye." types out below, then the 5-agent flow diagram from docs/diagrams/agent_flow.png slides in below. Hold for 2 seconds. Fade to credits with team names. Export as MP4. Use OpenCV with PIL for text rendering — no external dependencies.

AUDIT ANTIGRAVITY TRACE FOLDER FOR COMPLETENESS

Scan docs/antigravity-trace/ and produce a report of what's present and what's missing. For each day folder (day1 through day5), confirm we have: at least one Manager View screenshot, one Plan Artifact screenshot, one browser-recording or screen-capture file. For the final-highlights folder, confirm we have 8-12 curated images with captions. If anything is missing, list it. This audit runs once on Day 5 morning.

PRE-SUBMISSION CHECKLIST RUN

Verify the submission package is complete:

1. Backend /health endpoint returns 200 from Cloud Run.
2. APK is signed, downloadable from the public link in README.
3. Demo video is uploaded to YouTube as unlisted and the link is in README.
4. docs/antigravity-trace/FINAL.pdf exists and renders correctly.
5. The 50-example intent_examples.jsonl exists and the Intent Agent's eval accuracy is $\geq 90\%$.
6. README has all top-level sections filled in.
7. The GitHub repo is set to public OR all judge emails are added as collaborators.
8. The Cloud Run service has min instances = 0 (don't waste credits after submission).

Print a green/red checklist. Stop the submission if any red.

APPENDIX D

Failure modes & final submission checklist.

Hackathon teams lose more often to recurring failure patterns than to lack of talent. Here are the ten that kill projects, ranked by frequency, and the pre-submission checklist that catches them before they cost you points.

The ten failure modes

- 01 Faked Urdu support.** Mobile UI is in Urdu but the agent only really handles English. Judges will speak Urdu during finals; the agent will fall over. **Fix:** the 50-example dataset and the Intent Agent prompt iteration. Aim for 92%+ accuracy on the dataset.
- 02 Invisible agents.** The system uses agents but the UI doesn't show them reasoning. The 20%-weighted Agentic Reasoning score evaporates. **Fix:** the AgentCard UI with deliberate reveal timing (Section 06).
- 03 Antigravity trace as afterthought.** Team scrambles to take screenshots on Day 5; trace looks thin and forced. **Fix:** the daily screenshot checklist in Section 03. Start Day 1, not Day 5.
- 04 Demo runs on a simulator.** Judges can tell. The demo loses credibility. **Fix:** film on a real Android device. Charge it the night before.
- 05 Voice fails mid-demo.** Live voice fails at finals; team panics. **Fix:** if voice isn't reliable on three out of three test runs on Day 4, ship text-only. Add DEMO_MODE fallback (Appendix C).
- 06 Ugly receipt PDF.** The Booking Agent's receipt looks like a default ReportLab dump. Judges see it on screen for 2 seconds in the demo. **Fix:** design the receipt deliberately. Match the Bulao design tokens. Iterate until it could pass for Stripe.
- 07 Ranking reasoning sounds generic.** "Best option for you" instead of "Only 1.4km away, 4.8 rating, 47 jobs in your area." The agent-reasoning score takes a hit. **Fix:** the Ranking Agent prompt and its must-contain-a-digit validator.
- 08 README is a placeholder.** Judges open the repo, see TODO sections, mark down on the documentation/polish score. **Fix:** the README structure on Day 5 morning, fully filled.
- 09 Cloud Run service is down at finals.** Min-instances was set to 0 and the cold-start kills the demo. **Fix:** set min-instances to 1 for the finals presentation window; revert after.
- 10 Single point of failure for screen-recording.** Only one teammate has the recording, their laptop dies the night before submission. **Fix:** after each rehearsal, upload the recording to shared Drive immediately.

The submission day checklist

Run this in order on Day 5 afternoon. Don't submit until every box is checked.

Functional

- End-to-end demo works on the demo phone with three different Urdu inputs.
- Intent Agent eval accuracy $\geq 92\%$ on the 50-example dataset.
- Cloud Run service responds to /health with a 200.
- Receipt PDF renders and downloads from the Cloud Storage URL.
- Follow-up reminder fires correctly within the demo's accelerated timeline.
- Voice input works reliably (or has been removed from the demo).
- DEMO_MODE fallback path tested and known to work.

Documentation

- README has all 11 sections from Section 08 filled in.
- Architecture diagram and 5-agent diagram are embedded in README.
- Demo video uploaded to YouTube (unlisted), link in README.
- docs/antigravity-trace/FINAL.pdf exists with 8–12 captioned screenshots.
- Team names and roles listed in README.
- backend/data/intent_examples.jsonl is committed and linked from README.

Trace deliverable

- Manager View screenshot showing 3+ parallel agents (Day 1).
- Plan Artifact screenshot for a non-trivial feature.
- Browser recording of an Antigravity agent testing the app.
- Knowledge Base screenshot showing saved context for the agents.
- Diff with reviewer comments showing iterative refinement.
- One screenshot per build day in the FINAL.pdf.

Deployment & infra

- Cloud Run service set to min-instances=1 for the finals window.
- APK signed, uploaded to public link, link in README.
- Firestore has the 100 mock providers and at least 3 sample bookings.
- Cloud Storage bucket for receipts is publicly readable.
- GitHub repo is public OR judges added as collaborators.

Final submit

- Form filled in completely, all required fields.
- Screenshot the submission confirmation page.
- Confirm submission email received.
- Slack/WhatsApp the team. Celebrate. Sleep.

If every box above is checked, you have delivered an honest, complete, well-documented agentic system that solves a real Pakistani problem with visible reasoning. That is what wins this hackathon. The rest is presentation.